LABORATORY OBSERVATION

Course:	Full Stack Web Development
Course Code:	23CM4121
Experiment No.:	3
Student Name:	P.Raghu Ram
Roll No.:	A24126552102
Date:	30-08-2026

1. TITLE

Design and Implementation of Dynamic Student Profile Generator Using JavaScript ES6 Classes and DOM Manipulation

2. AIM

To develop an interactive Student Profile Generator web application using HTML5 form controls, CSS3 styling, JavaScript ES6 classes, event listeners, and dynamic DOM manipulation.

3. OBJECTIVE

The objectives of this experiment are:

- To design a styled user input form for capturing student academic metadata (Name, Roll Number, Department, CGPA).
- To implement JavaScript ES6 object-oriented class structure (class Student) for encapsulating student data.
- To register button click event listeners (addEventListener('click', ...)) for form interaction.
- To validate input fields and display alert warnings for incomplete entries.
- To dynamically construct and append HTML profile card elements (document.createElement, appendChild) into the DOM.

4. THEORY

JavaScript ES6 introduces modern object-oriented programming syntax through class definitions (class Student { constructor(...) { ... } }). Classes allow structured instantiation of objects with encapsulated properties such as name, roll number, department, and CGPA.

Document Object Model (DOM) manipulation allows dynamic updating of web content. Using document.getElementById(), document.createElement('div'), template literals (`...${...}`), innerHTML, and appendChild(), elements can be programmatically instantiated and attached to the page in response to user interaction.

Application Flow:

User Input -> Button Click Event Listener -> Field Validation -> ES6 Student Object Instantiation -> Dynamic DOM Element Creation -> Profile Card Appended to Webpage

5. ALGORITHM / PROCEDURE

1. Create an HTML5 document with title 'Student Profile Generator' and input form container.
2. Style the container, inputs, button, and dynamic student card using CSS3 (flexbox column, box-shadows, rounded borders).
3. Define an ES6 Student class with constructor accepting name, rollNumber, department, and cgpa.
4. Retrieve references for form input elements, submit button (generateBtn), and output container (profileOutput).
5. Attach a click event listener to the submit button using addEventListener.
6. Read input values and check if any input field is empty; show alert if validation fails.
7. Instantiate a new Student object using the 'new' keyword.
8. Create a new 'div' element (document.createElement('div')) and set its class to 'student-card'.
9. Use template literals to set card innerHTML with centered student name, roll number, department, and CGPA.
10. Append the newly created card element to outputContainer (appendChild) and clear all form input fields.

6. PROGRAM

index.html

```
<!DOCTYPE html>
<html lang="en">
<head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-scale=1.0">
    <title>Student Profile Generator</title>
    <style>
        body {
            font-family: Arial, sans-serif;
            background-color: #f4f4f9;
            margin: 50px;
            display: flex;
            flex-direction: column;
            align-items: center;
        }

        .form-container {
            background: white;
            padding: 20px;
            border-radius: 8px;
            box-shadow: 0 4px 8px rgba(0,0,0,0.1);
            width: 300px;
            display: flex;
            flex-direction: column;
            gap: 15px;
        }

        input {
            padding: 10px;
            border: 1px solid #ccc;
            border-radius: 4px;
            font-size: 14px;
        }

        button {
            padding: 10px;
            background-color: #28a745;
            color: white;
            border: none;
            border-radius: 4px;
            cursor: pointer;
            font-size: 16px;
        }

        button:hover {
            background-color: #218838;
        }

        /* Styling for the dynamically generated profile */
        #profileOutput {
            margin-top: 30px;
        }

        .student-card {
            background: #ffffff;
            border-left: 5px solid #007bff;
            padding: 20px;
            border-radius: 8px;
            box-shadow: 0 4px 8px rgba(0,0,0,0.1);
            width: 300px;
        }

        .student-card h3 {
            margin-top: 0;
            color: #333;
        }
    </style>
</head>
<body>

    <h2>Student Profile Generator</h2>
```

```
    <div class="form-container">
        <input type="text" id="nameInput" placeholder="Enter Student Name">
        <input type="text" id="rollInput" placeholder="Enter Roll Number">
        <input type="text" id="deptInput" placeholder="Enter Department">
        <input type="number" id="cgpaInput" placeholder="Enter CGPA" step="0.1">

        <button id="generateBtn">Create Profile</button>
    </div>

    <div id="profileOutput"></div>


    <script>

        class Student {
            constructor(name, rollNumber, department, cgpa) {
                this.name = name;
                this.rollNumber = rollNumber;
                this.department = department;
                this.cgpa = cgpa;
            }
        }

        const button = document.getElementById("generateBtn");
        const outputContainer = document.getElementById("profileOutput");

        // 3. Add an event listener to the button
        button.addEventListener("click", function() {
            const nameValue = document.getElementById("nameInput").value;
            const rollValue = document.getElementById("rollInput").value;
            const deptValue = document.getElementById("deptInput").value;
            const cgpaValue = document.getElementById("cgpaInput").value;

            if(nameValue === "" || rollValue === "" || deptValue === "" || cgpaValue === "") {
                alert("Please fill out all fields!");
                return;
            }


            const newStudent = new Student(nameValue, rollValue, deptValue, cgpaValue);

            outputContainer.innerHTML = "";


            const cardElement = document.createElement("div");

            cardElement.className = "student-card";
            cardElement.innerHTML = `
                <center><h3><b>${newStudent.name}</b></h3></center>
                <p><strong>Roll Number:</strong> ${newStudent.rollNumber}</p>
                <p><strong>Department:</strong> ${newStudent.department}</p>
                <p><strong>CGPA:</strong> ${newStudent.cgpa}</p>
            `;
            outputContainer.appendChild(cardElement);
            document.getElementById("nameInput").value = "";
            document.getElementById("rollInput").value = "";
            document.getElementById("deptInput").value = "";
            document.getElementById("cgpaInput").value = "";
        });
    </script>

</body>
</html>
```

7. CODE EXPLANATION

Code / Syntax	Explanation
`class Student { constructor(...) }`	ES6 class definition encapsulating student data fields into reusable object instances.
`button.addEventListener('click', ...)`	Binds an event handler function triggered when the 'Create Profile' button is clicked.
`document.getElementById(...)`	Accesses DOM input elements to extract user-entered form data values.
`document.createElement('div')`	Programmatically creates a new HTML <div> element in memory.
`cardElement.className = 'student-card'`	Assigns CSS class to style the dynamically created student profile card.
`` `...${newStudent.name}...` ``	Template literal formatting HTML markup dynamically with student object properties.
`outputContainer.appendChild(cardElement)`	Appends the dynamically constructed card element to the DOM output container.

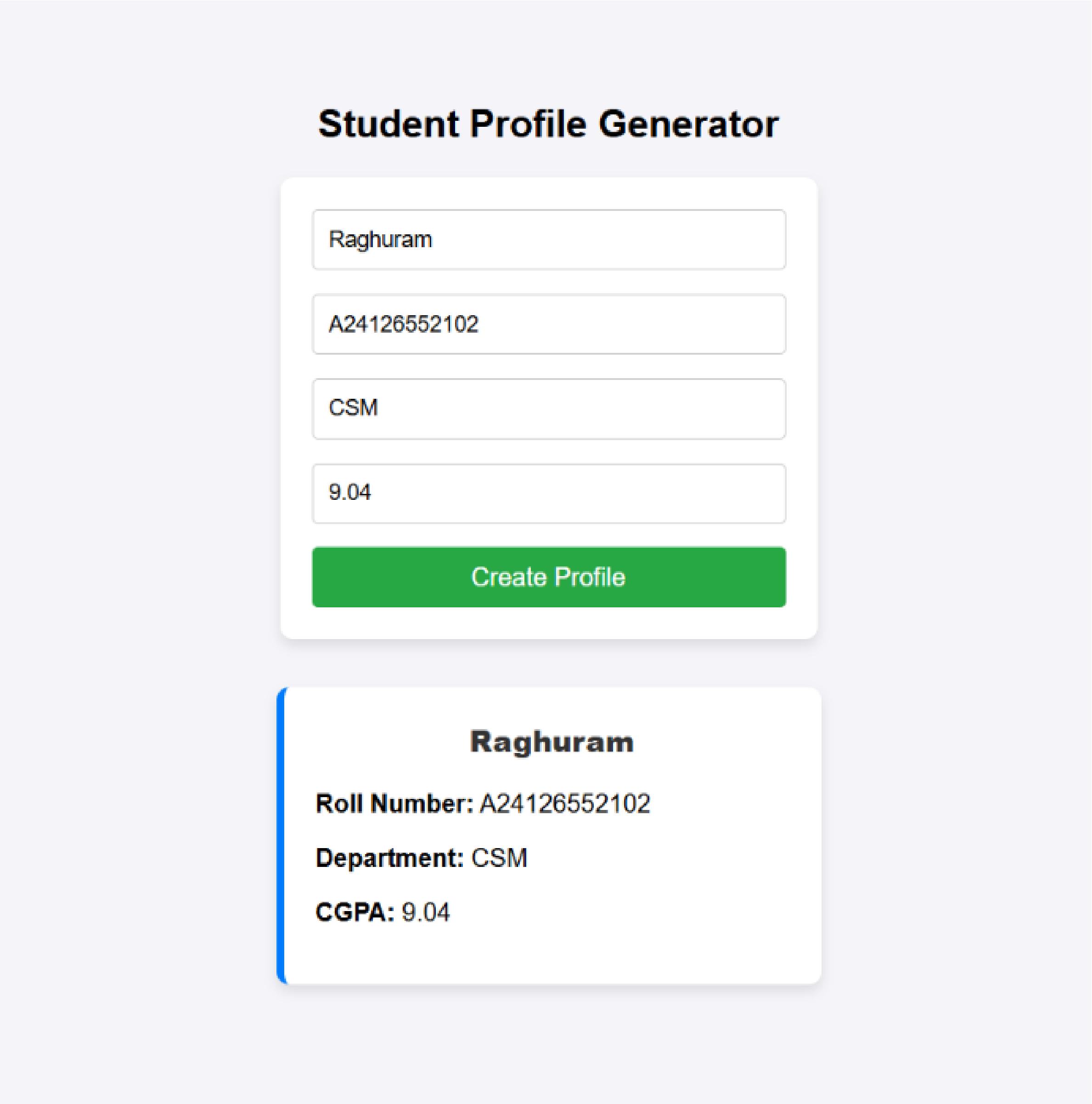
8. TEST CASES AND EXECUTION DATA

The experiment was executed with the following test cases to verify functionality:

Test Case	Input / Action	Expected Result	Actual Result	Status
TC-01	Form Rendering	Open page in browser	Form container with 4 inputs and green button renders	**Pass**
TC-02	Input Validation	Click button with empty fields	Browser alert 'Please fill out all fields!' is displayed	**Pass**
TC-03	Object Instantiation	Enter student details & click button	Student class object instantiated with provided data	**Pass**
TC-04	Dynamic Card Generation	Check #profileOutput container	Styled student card created with blue left border	**Pass**
TC-05	Form Reset	Inspect input fields after submission	Input fields cleared automatically for next entry	**Pass**

9. OUTPUT

Output 1: Student Profile Generator Form and Dynamically Generated Profile Card



10. RESULT

Thus, the Student Profile Generator web application utilizing ES6 classes, event listeners, and dynamic DOM manipulation was successfully designed, implemented, and verified.